

GrowthPulse LoRaWAN Bring-Up Guide

Status: engineering reference. The manual setup this guide describes is now automated by the app. Bringing a node up on LoRaWAN no longer means creating the TTS device, generating keys, pasting a payload formatter, or wiring a webhook by hand. The `provision-lorawan` function does all of that, the `lorawan-uplink` webhook decodes the raw bytes itself (no formatter), and `register-gateway` registers a gateway when you add it in the app. For the current step-by-step, see **GrowthPulse LoRaWAN Auto-Provision Setup**, **GrowthPulse Gateway Auto-Register Setup**, and the **Teammate / Web App Checklists**. This document is kept as the reference for how the LoRaWAN path works under the hood and for bench debugging.

How to take a GrowthPulse node from Wi-Fi to **LoRaWAN** end to end: node firmware, The Things Stack, gateway, and the uplink into the existing app. Researched against official sources (RadioLib, The Things Stack, The Things Network, Heltec, Semtech) in June 2026. Every nontrivial fact has a source at the bottom.

Hardware in hand: Heltec WiFi LoRa 32 V3 (ESP32-S3 + Semtech SX1262), US915. ThinkNode G1 gateway (G1-US915), Gateway EUI `E4:38:19:FF:FE:2A:58:80`.

The big picture: today's path is `Node -> Wi-Fi -> Losant -> Netlify -> app`. The LoRaWAN path is `Node (SX1262) -> ThinkNode G1 -> The Things Stack -> webhook -> our pipeline -> app`. The gateway and app stay; we add LoRa firmware, a network server account, and one webhook.

0. Recommended order of operations (manual bench bring-up)

In normal use you do none of this by hand; the app automates it (see the status note at the top). The sequence below is the manual path, useful for understanding the chain or debugging a node on the bench.

Bring-up fails when people try to do everything at once. Do this sequence:

1. **Stand up The Things Stack** (account, register the G1 gateway, create the application + one device). Section 3-4.
2. **Point the ThinkNode G1 at The Things Stack** and confirm it shows **Connected** in the console. Section 3.
3. **Flash the standalone LoRaWAN node sketch** (`firmware/GP_LoRaWAN/`) and get it to **join** and send an uplink you can see in the console. Section 2 + 6.
4. **Add the payload formatter** so the bytes decode into real fields. Section 5.
5. **Add the webhook** so uplinks land in the app pipeline. Section 5 + 7.

6. Merge the LoRaWAN path into the main firmware as a boot-selectable mode. This is now done; it ships as `GP_Combined`. Section 8.

Prove the radio join in isolation first. Merging into the combined firmware before the join works just makes debugging harder.

1. Library choice and why

Use **RadioLib** for the SX1262 LoRaWAN stack, with two helper libraries from the same well-known author (`ropg`):

Library	Role	Why
RadioLib (jgromes/ RadioLib, 7.x)	SX1262 driver + LoRaWAN Class A stack	First-class, native SX1262 LoRaWAN OTAA support. The de facto standard for this chip.
heltec_esp32_lora_v3 (ropg, v0.9.2)	Board wrapper (<code>heltec_unofficial.h</code>)	Hard-codes the V3 pin map and the TCXO/ RF-switch setup so you don't get them wrong.
LoRaWAN_ESP32 (ropg)	Session + DevNonce persistence	Saves nonces to flash so you don't re-join (and burn DevNonces) every boot.

Do NOT use MCCI LMIC. LMIC is an SX1272/SX1276 (SX127x) stack. It does not natively drive the SX1262's command interface (BUSY handshake, DIO1 interrupt, TCXO via DIO3, DIO2 RF switch). RadioLib's SX126x driver is purpose-built for this chip. Heltec's own LoRaWAN library works but is less documented; the community wrote the RadioLib wrapper specifically to avoid it.

Install via Arduino Library Manager: search "RadioLib", "Heltec ESP32 LoRa V3" (ropg), and "LoRaWAN ESP32" (ropg). Use the Espressif ESP32 Arduino core 2.0.16+ or 3.x.

The Heltec V3 SX1262 pin map (exact GPIOs)

These are the make-or-break numbers. Taken from the board's `heltec_unofficial.h` (matches Heltec's schematic):

Signal	GPIO
NSS / CS	8
DIO1 (interrupt)	14
RST / NRST	12
BUSY	13
SCK	9
MISO	11
MOSI	10

RadioLib constructor: `SX1262 radio = new Module(8, 14, 12, 13);` — order is **(NSS, DIO1, RST, BUSY)**.

Three V3 gotchas, all handled for you by `heltec_unofficial.h`:

- **TCXO on DIO3 (this is the #1 init failure).** The V3 uses a temperature-compensated oscillator powered from the SX1262's DIO3 pin. RadioLib must be told a **non-zero TCXO voltage** or the radio never starts (error `-707`). RadioLib's default of **1.6 V** works on the V3. (Some drivers use 1.8 V; both are reported working. The only hard rule: non-zero.)
- **DIO2 as RF switch.** The SX1262 drives its antenna TX/RX switch from DIO2; RadioLib enables this with `setDio2AsRfSwitch(true)`. Symptom if wrong: "TX succeeds but nothing is ever received."
- **DIO1 is labeled "DIO0" in Heltec's own files.** Ignore that; the SX1262's IO pins start at DIO1. The pin is GPIO14 either way.

The SX1262 and OLED are **not** on the Vext switched rail on the regular WiFi LoRa 32 V3, so the radio works without touching Vext (GPIO36). (The soil-sensor GPIO1 battery-ADC conflict from earlier is unrelated to the radio.)

2. OTAA, not ABP

Use **OTAA** (Over-The-Air Activation). The node carries a DevEUI, JoinEUI, and AppKey, performs a join handshake, and the network derives fresh session keys each join. It is the LoRaWAN-compliant, secure choice for a shipping product. ABP (hard-coded session keys) skips the join but is fragile (frame-counter desync, no key rotation) and is only for special cases. The Things Stack defaults to OTAA.

3. The Things Stack: account, cluster, and the gateway

"The Things Stack Community Edition" is now called **The Things Stack Sandbox** (same free TTN-hosted network; both names appear in docs).

Cluster: for North America use **nam1**. - Console: `https://nam1.cloud.thethings.network/console` - Quirk: you log in / create your account through **eu1** (the identity server lives there), but you manage gateways and devices on the **nam1** console.

3a. Register the ThinkNode G1 gateway

1. nam1 Console -> **Gateways** -> **+ Register gateway**.
2. **Gateway EUI:** paste the 16-hex EUI from your own gateway's label (the **EUI:** line), Confirm. (Our dev gateway is `E4:38:19:FF:FE:2A:58:80` ; whoever does the setup uses the gateway they physically have.)
3. **Gateway ID:** a unique lowercase id (convention `eui-e43819fffe2a5880`).
4. **Frequency plan: United States 902-928 MHz, FSB 2 (used by TTN)**. This is sub-band 2: 125 kHz uplink channels 8-15 plus 500 kHz channel 65. The gateway AND every device must use this exact plan.
5. **Register gateway**.

3b. Point the G1 at The Things Stack (Semtech UDP)

The G1 ships with the **Semtech UDP packet forwarder**, which is the simplest reliable path. In the G1's web UI (join its AP or LAN, browse `192.168.1.1` , login `root / root`):

1. **LoRaWAN -> LoRa Gateway**, set **Mode = Packet Forward**.
2. **Packet Forwarder Settings:** - Gateway EUI: auto-filled. - **Server Address:** `nam1.cloud.thethings.network` - **Server Port Up / Down:** `1700 / 1700`
3. **Channel Plan: US915** (sub-band 2 to match).
4. **Save & Apply**, restart the gateway.

Then in the TTS Console, open the gateway -> it should show **Connected**, and **Live data** should show traffic. (Basics Station with TLS is the upgrade path later; UDP is fine for bring-up.)

Note: `router.us.thethings.network:1700` from old tutorials is the **legacy V2** address. Use `nam1.cloud.thethings.network:1700` .

4. The Things Stack: application + device (OTAA keys)

Automated now: `provision-lorawan` creates the end device and its OTAA keys for you and pushes them to the board. The steps below are the manual equivalent, kept for reference and debugging.

4a. Create the application

Console -> **Applications** -> **+ Create application** -> give it an **Application ID** (lowercase, e.g. `growthpulse`) -> Create.

4b. Register the node as an end device (manual OTAA)

Application -> **End devices** -> **+ Register end device** -> **Enter end device specifics manually:**

- **Frequency plan:** United States 902-928 MHz, **FSB 2** (must match the gateway).
- **LoRaWAN version: MAC 1.0.4.** (RadioLib uses a monotonically-increasing DevNonce, which is 1.0.4/1.1 behavior — matching 1.0.4 avoids confusion. 1.0.3 also works but pair the regional-params version to it.)
- **Regional Parameters: RP002 Regional Parameters 1.0.4** (pair with the MAC version).
- **Activation mode: OTAA.**
- **JoinEUI / AppEUI:** a DIY node has none, so enter all zeros (`0000000000000000`) or a random value — then flash the SAME value into firmware.
- **DevEUI:** click **Generate**, then flash that exact value into firmware.
- **AppKey:** click **Generate**, then flash the same AppKey into firmware. (Only MAC 1.1 needs a separate NwkKey; for 1.0.x set the firmware's `nwkKey` equal to `appKey`.)
- **End device ID:** unique, convention `eui-<devEUI>`.
- **Register end device.**

The three values that must match on both sides: **DevEUI, JoinEUI, AppKey**. For a custom node they're generated in the console and copied into firmware.

4c. Dev-loop setting you will need

Because RadioLib's DevNonce increments and resets to a low value when you reflash, repeated flashing causes **"DevNonce has already been used"** rejections. For development, turn on the device's **General settings -> Join settings -> "Resets join nonces"**. (Production fix is to persist nonces in flash, which `LoRaWAN_ESP32` does — see Section 6.)

5. Uplink payload: byte layout, decoder, and webhook

Automated now: the `lorawan-uplink` webhook decodes the 9 raw bytes itself, so no TTS payload formatter is required, and the webhook and its token are already configured on the production side. The formatter and webhook steps below are the manual/reference version (the byte layout in 5a is still exactly what the firmware sends).

LoRaWAN frames are tiny (see Section 6 limits), so the node sends a packed binary payload, not JSON. The Things Stack decodes it with a JavaScript **payload formatter**, then a **webhook** POSTs the decoded JSON to our endpoint.

5a. The 9-byte uplink layout (fits every US915 data rate)

Send on `f_port = 2`. Temperatures are $^{\circ}\text{F} \times 10$ as signed int16 (one decimal, handles sub-freezing).

Bytes	Field	Encoding	Decode
0-1	soilTemperatureF	int16 BE, x10	raw/10
2-3	airTemperatureF	int16 BE, x10	raw/10
4	airHumidity	uint8 (0-100)	as-is
5-6	soilRaw	uint16 BE (0-4095)	as-is
7	soilMoisturePercent	uint8 (0-100)	as-is
8	flags	uint8 (battery/version, optional)	as-is

9 bytes is well under the DR0 limit of 11 bytes, so it transmits at the slowest (longest-range) rate.

5b. The Things Stack uplink decoder (paste into Application -> Payload formatters -> Uplink -> Custom JavaScript)

```
function decodeUplink(input) {
  var b = input.bytes;
  var warnings = [], errors = [];
  if (b.length < 8) {
    return { data: {}, warnings: warnings, errors: ["payload too short (need >= 8 bytes)"] };
  }
  function s16(hi, lo) { var v = (hi << 8) | lo; return v & 0x8000 ? v - 0x10000 : v; }
  var data = {
    soilTemperatureF:    s16(b[0], b[1]) / 10,
    airTemperatureF:    s16(b[2], b[3]) / 10,
    airHumidity:         b[4],
    soilRaw:             (b[5] << 8) | b[6],
    soilMoisturePercent: b[7]
  };
  if (b.length >= 9) data.flags = b[8];
  if (data.airHumidity > 100) warnings.push("airHumidity > 100");
  if (data.soilMoisturePercent > 100) warnings.push("soilMoisturePercent > 100");
  if (data.soilRaw > 4095) warnings.push("soilRaw exceeds 12-bit");
  return { data: data, warnings: warnings, errors: errors };
}
```

5c. The webhook

Application -> **Integrations** -> **Webhooks** -> + **Add webhook** -> **Custom webhook**: - Format: **JSON** - Base URL: `https://growthpulsecloud.com/.netlify/functions/lorawan-uplink` - **Uplink message** path: ``` (leave empty so it posts to the base URL) or `/` - Add a header X-Webhook-Token: `and put the same secret in the Netlify env var LORAWAN_WEBHOOK_TOKEN`` so only TTS can call the endpoint.

The webhook function (`netlify/functions/lorawan-uplink.js`) is in the repo. It verifies the token, reads `uplink_message.decoded_payload` and `end_device_ids.device_id`, and forwards the reading into the same pipeline the app already reads. The uplink JSON TTS posts looks like:

```
{
  "end_device_ids": { "device_id": "growthpulse-node-01", "dev_eui":
"70B3D57ED0001234" },
  "received_at": "2026-06-07T15:04:05Z",
  "uplink_message": {
    "f_port": 2, "f_cnt": 145, "frm_payload": "AQID...",
    "decoded_payload": { "soilTemperatureF": 68.4, "airTemperatureF": 74.1,
"airHumidity": 53, "soilRaw": 2048, "soilMoisturePercent": 42 },
    "rx_metadata": [ { "gateway_ids": { "gateway_id": "eui-e43819fffe2a5880" }, "rssi":
-57, "snr": 9.5 } ],
    "settings": { "data_rate": { "modulation": { "lorawan": { "spreading_factor": 10,
"bandwidth": 125000 } } }, "frequency": "903900000" }
  }
}
```

6. Airtime limits and how often the node may send

This is the single biggest mindset shift from Wi-Fi. LoRaWAN is not a streaming link.

- **TTN Fair Use Policy (the free Sandbox): 30 seconds of uplink airtime per device per day, and 10 downlink messages per device per day.**
- **US915 max application payload per data rate:** DR0 (SF10/125 kHz) = **11 bytes**; DR1 = 53; DR2 = 125; DR3/DR4 = 222.
- Our 9-byte frame at the worst rate (DR0) is ~0.3-0.4 s of airtime. Against 30 s/day that's roughly 75-100 messages/day, i.e. **one uplink every 15-30 minutes** with margin. **15 minutes is the recommended interval** for a battery plant sensor.

So the app must treat LoRaWAN devices very differently from Wi-Fi: the staleness window for a LoRaWAN node is already 15 minutes in the app (vs 45 s for Wi-Fi). Do not try to send every few seconds; you'd blow the daily airtime in minutes, LoRa's DR0 is only ~980 bit/s, and you'd crowd the shared spectrum.

7. Downlinks / commands (e.g. factoryReset) over LoRaWAN

RadioLib is **Class A only**, which is correct for a battery sensor. The consequence: a downlink can only reach the node in the two short RX windows **right after the node uplinks**. So a queued command waits until the node's next check-in.

- **Latency:** worst case = your uplink interval. With 15-minute uplinks, a `factoryReset` could take up to ~15 minutes to arrive. Design the command UX as "queued, applies on next check-in," not instant.
- **Schedule a downlink:** TTS Console -> device -> **Messaging -> Downlink** (pick an FPort 1-223, hex bytes, confirmed or not). Also available via CLI (`ttn-lw-cli end-devices downlink push`) or the webhook API (`.../down/push` with base64 `frm_payload`).
- **Receive in firmware:** the downlink comes back as the return of `node.sendReceive(...)` (the same call that sends the uplink). Return value: **1** = downlink in RX1, **2** = in RX2, **0** = none, **<0** = error. Read the bytes and the `fPort` from the event struct, then dispatch (e.g. FPort 10 = `factoryReset`).

8. Combined Wi-Fi + LoRaWAN firmware (implemented as GP_Combined)

This is built and shipping as `firmware/GP_Combined/GP_Combined.ino` , with the app's transport picker driving the switch. The design notes below explain how it works.

One ESP32-S3 firmware can hold **both** stacks and pick one at boot. The two radios are independent (on-die 2.4 GHz Wi-Fi vs external SX1262 sub-GHz), but a sensor only ever uses one path per deployment, so we choose at boot and never bring up the other.

Storage / flash: the Heltec V3 has 8 MB flash. Both stacks compiled together fit fine; the constraint is the per-app-partition size. If the combined image overflows the default app slot, set Arduino **Tools -> Partition Scheme** to a larger-app layout. Keep a dual-OTA scheme if you want Wi-Fi OTA self-update; only drop to "Huge App / No OTA" if you must.

Mode selector in NVS (Preferences):

```

#include <Preferences.h>
Preferences prefs;

String getMode() {
    prefs.begin("netcfg", true);          // read-only
    String m = prefs.getString("mode", "wifi"); // default wifi
    prefs.end();
    return m;
}
void setMode(const String& m) {
    prefs.begin("netcfg", false);
    prefs.putString("mode", m);          // "wifi" | "lorawan"
    prefs.end();
}
void switchTo(const String& m) { setMode(m); delay(100); ESP.restart(); }

void setup() {
    if (getMode() == "lorawan") startLoRaWAN(); // RadioLib path
    else                        startWiFi();    // current Wi-Fi/MQTT path
}

```

This is the moment the app's transport picker becomes real: switching a device to "LoRaWAN" in the app would queue a downlink (or be set during provisioning) that calls `switchTo("lorawan")`, and the node reboots into LoRaWAN mode. Until then the app honestly shows Wi-Fi.

Cautions: RAM/heap is the real ceiling, not flash, so initialize only the chosen stack. Never run both networking paths at once. Wi-Fi draws far more power than duty-cycled LoRa, so the LoRaWAN path is the battery path and the Wi-Fi path implies USB/AC power.

9. Troubleshooting (ordered by most likely)

Symptom	Likely cause	Fix
<code>radio.begin()</code> returns -7071-706	TCXO voltage wrong (V3 has a TCXO)	Use a non-zero TCXO voltage (1.6 V default). Use <code>heltec_unofficial.h</code> which sets it.
<code>radio.begin()</code> returns -2 (CHIP_NOT_FOUND)	Wrong NSS/BUSY/RST pin or SPI	Confirm pins 8/14/12/13, SCK9/MISO11/MOSI10
Nothing in gateway Live data	Sub-band wrong, or range/antenna	Use subBand 2 (<code>LoRaWANNode node(&radio,&US915,2)</code>); separate node and gateway by 5-10 m with a wall
Join request in gateway Live data but not device Live data	DevEUI/JoinEUI/AppKey mismatch, or FSB mismatch	Re-check the three keys match firmware exactly; both sides on FSB2
Gateway sends join-accept, device never joins	RX-window timing / slow gateway backhaul / too close	Check gateway latency; move apart; use library TCXO defaults
Joins once, then -1116 "DevNonce already used"	DevNonce reset by reflash	Dev: enable "Resets join nonces" in TTS. Prod: persist nonces (LoRaWAN_ESP32)
Re-joins every boot, burns airtime	No session/nonce persistence	Use <code>LoRaWAN_ESP32</code> to save nonces to flash, session to RTC RAM
<code>-1101 SESSION_DISCARDED</code> after power-cycle	Expected (RTC RAM wiped on cold boot)	Benign; nonces in flash let it re-join cleanly

Debug procedure: keep the TTS **gateway Live data** tab open while the node transmits. See the join request there? Gateway is fine, problem is keys/sub-band. See nothing? Radio/gateway/range problem. Enable `RADIOLIB_DEBUG_PROTOCOL` and confirm the serial log prints "subband 2" with channels 8-15.

Sources

Library + board: - RadioLib SX1262 class (begin signature, TCXO default 1.6 V, error codes): https://jgromes.github.io/RadioLib/class_s_x1262.html - RadioLib LoRaWAN_Starter example + notes (OTAA flow, subBand 2, debugging): https://github.com/jgromes/RadioLib/blob/master/examples/LoRaWAN/LoRaWAN_Starter/notes.md - ropg/heltec_esp32_lora_v3 (pin map, DIO1/DIO0, Vext, RadioLib): https://github.com/ropg/heltec_esp32_lora_v3 - ropg/LoRaWAN_ESP32 (nonce/session persistence): https://github.com/ropg/LoRaWAN_ESP32 - Heltec WiFi LoRa 32 V3 (SX1262, TCXO, 8 MB flash): <https://heltec.org/project/wifi-lora-32-v3/>

The Things Stack: - Adding gateways: <https://www.thethingsindustries.com/docs/hardware/gateways/concepts/adding-gateways/> - Semtech UDP packet forwarder: <https://www.thethingsindustries.com/docs/hardware/gateways/concepts/udp/> - Addresses (nam1 cluster): <https://www.thethingsindustries.com/docs/concepts/ttn/addresses/> - Adding devices manually / OTAA: <https://www.thethingsindustries.com/docs/hardware/devices/adding-devices/manual/otaa/> - Webhooks (creating + format): <https://www.thethingsindustries.com/docs/integrations/webhooks/creating-webhooks/> - Uplink payload formatters (decodeUplink): <https://www.thethingsindustries.com/docs/integrations/payload-formatters/javascript/uplink/> - Scheduling downlinks: <https://www.thethingsindustries.com/docs/integrations/webhooks/scheduling-downlinks/> - Troubleshooting devices (DevNonce, FSB, join-accept timing): <https://www.thethingsindustries.com/docs/hardware/devices/troubleshooting/> - ThinkNode G1 network config (Elecrow wiki): https://www.elecrow.com/wiki/Quick_Network_Configuration_of_Gateway-ThinkNode_Gateway_and_LR1262_Module.html

The Things Network: - US915 regional parameters + payload sizes: <https://www.thethingsnetwork.org/docs/lorawan/regional-parameters/us915/> - Duty cycle + Fair Use Policy (30 s/day, 10 downlinks/day): <https://www.thethingsnetwork.org/docs/lorawan/duty-cycle/> - Device classes (Class A RX windows): <https://www.thethingsnetwork.org/docs/lorawan/classes/>

ESP32: - Preferences (NVS): <https://docs.espressif.com/projects/arduino-esp32/en/latest/tutorials/preferences.html> - Partition tables: https://docs.espressif.com/projects/arduino-esp32/en/latest/tutorials/partition_table.html

GrowthPulse Engineering, internal. LoRaWAN bring-up reference.